



Universidad Rafael Landívar

Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Manejo e Implementación de Archivos

Catedrático: Ingeniero Luis Enrique

Laboratorio 1 – Semana 14

Diego Alessandro Lopez Higueros – 1187525

10/7/2026



Nombre: Diego Alessandro Lopez Higueros **Carné:** 1187525

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

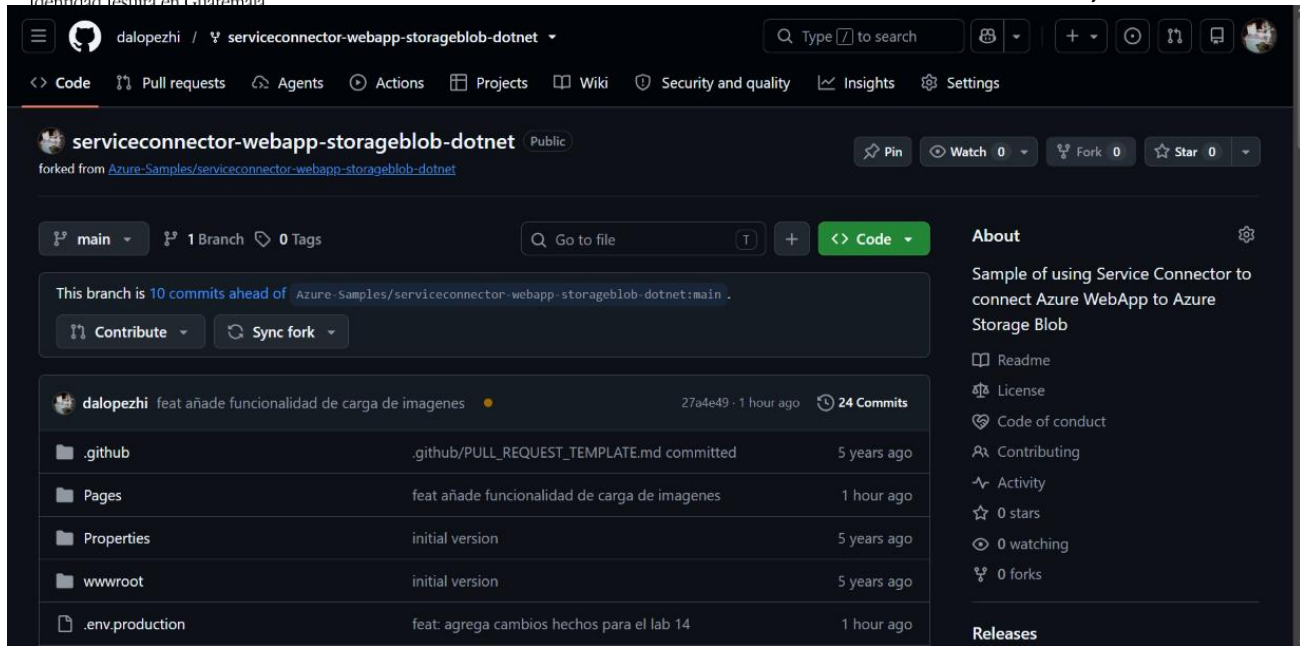
Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable



2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

3. Crear DotNet Env

a) En terminal, instalar: **dotnet add package DotNetEnv**

b) Crear .env file en raíz de proyecto, valores ->

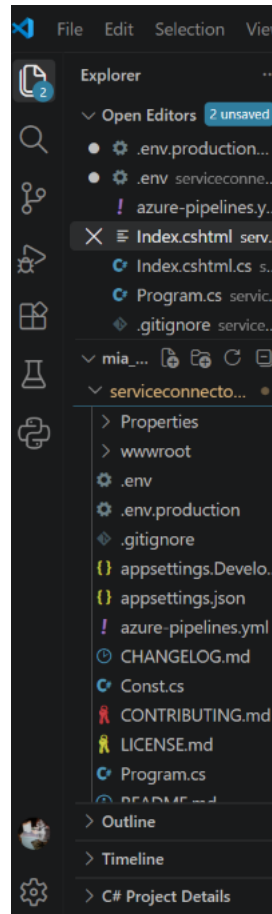
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/

AZURE_STORAGE_CONTAINER_NAME=default

c) Crear .env.production con los mismos valores en el mismo lugar

d) Anadir cambio brindado en program.cs

e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run



4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

Microsoft Azure

Search resources, services, and docs (G+)

Home > Storage center | Blob Storage > miastorage202601

miastorage202601 | Containers

Storage account

cont

+ Add container ↑ Upload ↻ Refresh 🗑 Delete 🔒 Change access level 🔄 Restore containers Edit columns

Search containers by prefix

Only show active containers

Showing all 4 items

☐	Name	Last modified	Anonymous access level	Lease state
☐	\$logs	9/23/2026, 9:07:51 AM	Private	Available
☐	archivos	10/2/2026, 9:48:41 AM	Private	Available
☒	default	10/7/2026, 7:56:48 AM	Private	Available
☐	documentos	9/23/2026, 9:13:35 AM	Private	Available

Add or remove favorites by pressing Ctrl+Shift+F



Microsoft Azure

Search resources, services, and docs (G+)

Home > Storage center | Blob Storage > miastorage202601 | Containers

default Container

Search

+ Add Directory ↑ Upload 🔒 Change access level ↻ Refresh 🗑 Delete 📄 Copy 📄 Paste 🔄 Rename 🔑 Acquire lease ...

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

default

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 2 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	HelloWorld	10/7/2026, 8:21:49 PM	Hot (Inferred)	Block blob	62 B	Available	...
☐	Huffman_Te...	10/7/2026, 9:49:34 PM	Hot (Inferred)	Block blob	966.07 KiB	Available	...

Add or remove favorites by pressing Ctrl+Shift+F

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → tu Storage Account → Data storage → Containers
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)
- Configurar la conexión entre App Service y Blob Storage: **App Service** → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create.
- Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`
- Verifique que pueda obtener un token para Azure Storage: `az account get-access-token --resource https://storage.azure.com/`
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> `dotnet run`
- Si hay error en acceso, ejecutar en terminal de vs code: `az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"`

Obtener info: `az account show --query "{subscriptionId:id, user:user.name}" -o table`

`az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table`

- Cerrar process `ctrl + c` y logear de nuevo: `az logout` y `az login`, Por ultimo `dotnet run`



```
PS C:\Users\50257\mia_semana14\serviceconnector-webapp-storageblob-dotnet> az --version
azure-cli                                2.91.0

core                                    2.91.0
telemetry                                1.1.0

Dependencies:
msal                                     1.39.0
azure-mgmt-resource                     24.0.0

Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\50257\.azure'
Extensions directory 'C:\Users\50257\.azure\cliextensions'

Python (Windows) 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)]

Legal docs and information: aka.ms/AzureCliLegal

Your CLI is up-to-date.
PS C:\Users\50257\mia_semana14\serviceconnector-webapp-storageblob-dotnet> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

ts: WebStorageSample  Debug Any CPU  C# Workspace Dev Mode  Tab Moves Focus  Ln 1, Col 1  Spaces: 4  UTF-8  CRLF  {} Dotenv  [Icons]

Your CLI is up-to-date.
PS C:\Users\50257\mia_semana14\serviceconnector-webapp-storageblob-dotnet> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure for Students    d15d9e4e-b64a-472d-9d71-d8c655de4aea  Universidad Rafael Landivar

The default is marked with an *; the default tenant is 'Universidad Rafael Landivar' and subscription is 'Azure for Students' (d15d9e4e-b64a-472d-9d71-d8c655de4aea).

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Universidad Rafael Landivar
Subscription: Azure for Students (d15d9e4e-b64a-472d-9d71-d8c655de4aea)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default

ts: WebStorageSample  Debug Any CPU  C# Workspace Dev Mode  Tab Moves Focus  Ln 1, Col 1  Spaces: 4  UTF-8  CRLF  {} Dotenv  [Icons]
```



Nota: no olvidar subir cambios a repo github fork

Colocar en el archivo lo siguiente:

- main

```
vmImage: 'windows-latest'
```

dotnetVersion: '9.x'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

```
version: '$(dotnetVersion)'
```

```
projects: '**/*.csproj'
```



3. Configurar ambiente de producción

```
- powershell: |  
  Copy-Item ".env.production" ".env" -Force  
  displayName: 'Configurar .env de producción'
```

4. Compilar

```
- task: DotNetCoreCLI@2  
  displayName: 'Compilar aplicación'  
  inputs:  
    command: 'build'  
    projects: '**/*.csproj'  
    arguments: '--configuration $(buildConfiguration) --no-restore'
```

5. Publicar

```
- task: DotNetCoreCLI@2  
  displayName: 'Publicar aplicación'  
  inputs:  
    command: 'publish'  
    projects: '**/*.csproj'  
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'  
    zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1  
  displayName: 'Desplegar en Azure App Service'  
  inputs:  
    azureSubscription: '$(azureSubscription)'  
    appType: 'webApp'  
    appName: '$(appName)'  
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```



The screenshot shows the Azure DevOps web interface. At the top, the user 'dalopezhi' is logged in, and the repository 'serviceconnector-webapp-storageblob-dotnet' is selected. The 'main' branch is active, and the file 'azure-pipelines.yml' is open. The pipeline definition is as follows:

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: 'windows-latest'
6
7 variables:
8   buildConfiguration: 'Release'
9   dotnetVersion: '9.x'
10
11 # Completar posteriormente
12 azureSubscription: 'MIA_Connection'
13 appName: 'WebAppMIA2026'
14
15 steps:
16
17 # 1. Instalar .NET 6
```

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps con misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure: Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save



MIA

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Library
- Test Plans
- Artifacts
- Project settings

Pipelines

Recent All Runs

New pipeline

Filter pipelines

Recently run pipelines

Pipeline	Last run
MIA-blobconnector-ci-cd	#20261008.5 • feat añade funcionalidad de c... Manually run by main 51m ago 3m 10s

mia-2026 / MIA / Settings / Service connections

Search

Menu Icons

Project Settings

MIA

- General
- Overview
- Teams
- Permissions
- Notifications
- Service hooks
- Dashboards
- Boards
- Project configuration
- Team configuration
- GitHub connections
- Pipelines
- Agent pools

Service connections

Security

New service connection

Filter by keywords

Created by

Type

Authentication method

Name	Type	Authentication method
dalopezhi	GitHub	Azure Pipelines App
MIA_Connection	Azure Resource Manager	Workload Identity federation (Automatic)

8. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name

appName: Tu_app_name

- Al hacer commit, verificar el pipeline se ejecuta automáticamente

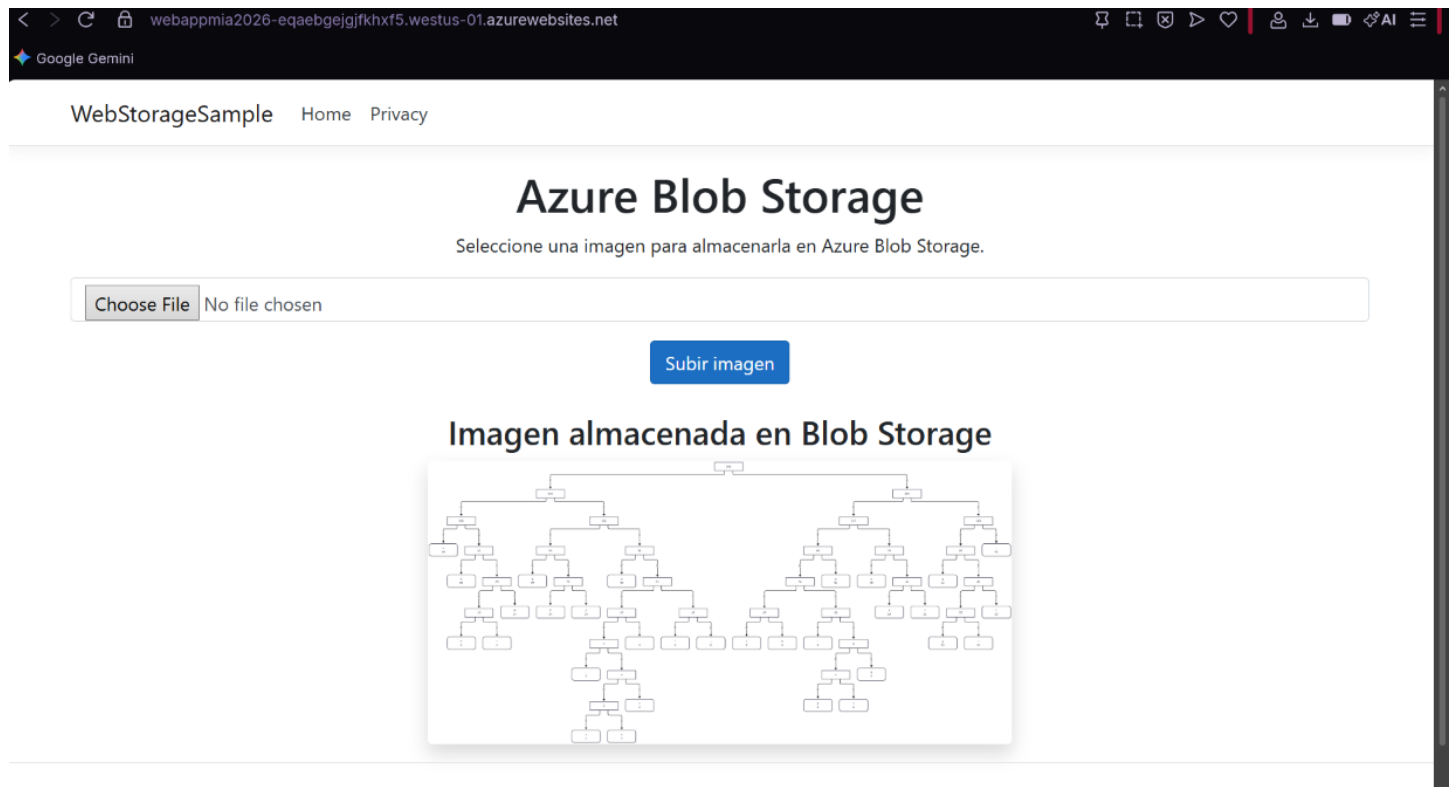


```
1 trigger:
2   - main
3
4 pool:
5   vmImage: 'windows-latest'
6
7 variables:
8   buildConfiguration: 'Release'
9   dotnetVersion: '9.x'
10
11 # Completar posteriormente
12 azureSubscription: 'MIA_Connection'
13 appName: 'WebAppMIA2026'
14
15 steps:
```

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios





Microsoft Azure

Search resources, services, and docs (G+/)

Home > Storage center | Blob Storage > miastorage202601 | Containers

default Container

Search

+ Add Directory ↑ Upload 🔒 Change access level ↻ Refresh 🗑 Delete 📄 Copy 📄 Paste 🔄 Rename 🔗 Acquire lease ⋮

default

Authentication method: Access key (Switch to Microsoft Entra user account)

🔍 Add filter

🔍 Search blobs by prefix (case-sensitive) Only show active blobs

Showing all 2 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	📄 HelloWorld	10/7/2026, 8:21:49 PM	Hot (Inferred)	Block blob	62 B	Available	⋮
☐	📄 Huffman_Te...	10/7/2026, 9:49:34 PM	Hot (Inferred)	Block blob	966.07 KiB	Available	⋮

Add or remove favorites by pressing Ctrl+Shift+F

10. Preguntas de analisis

- ¿Para qué sirve el archivo .env en la aplicación?
Sirve para almacenar variables de entorno y configuraciones sensibles (como endpoints de Azure y nombres de contenedores) fuera del código fuente. Esto permite cambiar los parámetros según el entorno (desarrollo o producción) sin alterar el programa y evita subir credenciales expuestas al repositorio.
- ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
Porque App Service es un entorno sin estado (*stateless*) donde los archivos guardados de forma local se borran cuando el servidor se reinicia, escala o despliega una nueva versión. Blob Storage ofrece almacenamiento persistente, altamente escalable y accesible de forma independiente sin saturar el disco o el servidor web.
- ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
El endpoint de Blob Storage es la dirección específica de la cuenta de almacenamiento en la nube destinada a guardar y servir archivos o datos binarios. En cambio, la URL de App Service es el dominio público donde está alojada y ejecutándose la interfaz de la aplicación web.
- ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
Porque elimina la necesidad de manejar, rotar o incrustar contraseñas y claves maestras en el código o en archivos de configuración. Azure administra automáticamente las credenciales mediante asignación de roles (RBAC), lo que mejora la seguridad y simplifica la autenticación entre servicios de Azure.
- ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
GitHub aloja el código fuente y el historial de cambios con control de versiones. Azure DevOps administra la canalización CI/CD para compilar, probar y desplegar automáticamente la aplicación. La Service Connection es el enlace seguro que autoriza a Azure DevOps para realizar despliegues dentro de la suscripción de Azure sin exponer credenciales.
- ¿Qué ocurre después de hacer push a main?



Se dispara un trigger en GitHub que notifica a Azure DevOps para iniciar la ejecución del pipeline. La plataforma levanta un agente virtual, descarga el código reciente, restaura dependencias, compila, genera el paquete ejecutable y lo despliega automáticamente en Azure App Service.

7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
Revisaría los permisos de acceso y roles de Azure. Específicamente, verificaría que la Identidad Administrada de App Service (o el usuario activo) tenga asignado el rol "Storage Blob Data Contributor" en la cuenta de almacenamiento, además de comprobar que las reglas de red o el firewall de la Storage Account no estén bloqueando la conexión.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.